



FORMATION IA · MODULE 2

Python pour l'IA

Aucun rappel théorique long : on pratique, on corrige, on avance.

Plan

- 1 Variables
- 2 Structures de données
- 3 Fonctions
- 4 Conditions
- 5 Boucles
- 6 Manipulation des fichiers
- 7 Introduction à NumPy
- 8 Introduction à Pandas



Python — Résumé de la syntaxe

Les bases pour bien commencer

1 Règles importantes

- Pas de point-virgule (;) en fin de ligne
- Indentation obligatoire (4 espaces)
- Sensible à la casse : age ≠ Age

```
if age >= 18:
    print("Majeur")
```

2 Variables & types

- Pas besoin de déclarer le type

```
age = 18      # int
prix = 3.14   # float
nom = "Ali"   # str
```

- Aussi : bool, list, dict

3 Affichage & saisie

```
print("Bonjour")
nom = input("Nom : ")
age = int(
    input("Âge : ")
)
```

- int() convertit la saisie en entier

4 Commentaires

- Sur une ligne :

```
# ceci est un commentaire
```

5 Opérateurs

Arithmétiques

```
+ - * / // % **
```

Comparaison

```
== != < > <= >=
```

Logiques

```
and or not
```

Affectation : = += -= *= /=

À retenir

- ★ Indentation (4 espaces) = structure du programme
- ★ Python déduit automatiquement le type des variables
- ★ Toujours ':' après if, elif, else, for, while, def
- ★ La lisibilité du code compte autant que son fonctionnement

L'environnement de travail



Le notebook

Un fichier (.ipynb) découpé en cellules, exécutées une par une.



Cellule de code

Contient des instructions Python. Exécution avec Maj + Entrée.



Cellule Markdown

Pour documenter, expliquer et structurer le notebook.



Exécution incrémentale

On relance une cellule sans tout ré-exécuter.

Variables

En Python, une variable sert à stocker une valeur.



Une variable est une boîte étiquetée : `nom = valeur`



Pas besoin de déclarer le type à l'avance
Python le déduit automatiquement.



Le nom doit être clair, sans espace
Exemple : `age_client`



Python est sensible à la casse
`age` et `Age` sont différents.

Exemple

```
# Création de variables
tokens = 42
prompt_user = "Un chat"
score_confiance = 0.98

# Utilisation
tokens = tokens + 1
print(tokens) # Affiche : 43
```

Exercice

- 1 Créer une variable `nom` avec votre prénom
- 2 Créer une variable `age` avec votre âge
- 3 Créer une variable `est_majeur` (booléen)
- 4 Afficher les trois variables avec `print()`



Structures de données

Python propose plusieurs structures pour stocker et organiser des données.



Liste []

Valeurs ordonnées et modifiables, indexées à partir de 0.



Exemple : `notes = [12, 15, 9]`

Une liste de notes d'examen.



Dictionnaire {}

Associe des clés à des valeurs.



Exemple : `eleve = {"nom": "Ada", "note": 17}`

Informations d'un élève.



Tuple ()

Comme une liste, mais figée (non modifiable).



Exemple : `coordonnees = (36.8, 10.2)`

Coordonnées géographiques (latitude, longitude).



En IA

Un dataset est souvent une liste de dictionnaires.



Exemple : `dataset = [{"nom": "Ada", "note": 17}, {"nom": "Bob", "note": 15}]`

Liste d'enregistrements (plusieurs élèves).



Exercice

- 1  Créer une liste `scores` avec les valeurs `[0.65, 0.80, 0.75, 0.90]`.
- 2  Ajouter le score `0.85` à la fin de la liste `scores`.
- 3  Supprimer le premier score de la liste `scores`.
- 4  Créer un dictionnaire `eleve` avec les clés : `nom`, `age`, `note` et les valeurs : `"Sara"`, `20`, `16.5`.
- 5  Créer un dataset `classe` contenant deux élèves (dictionnaires) dans une liste.



Fonctions

**Définition :**

Une fonction est un bloc de code qui effectue une tâche précise.

**Entrées (paramètres) :**

Les paramètres permettent de donner des valeurs à la fonction.

**Résultat (return) :**

La fonction renvoie un résultat que l'on peut utiliser ensuite.

**Appel d'une fonction :**

On utilise le nom de la fonction suivi de () avec les valeurs à fournir.

Exemple

```
def carre(nombre):  
    return nombre ** 2  
  
resultat = carre(5)  
print(resultat)    # 25
```

- ➔ `carre(5)` appelle la fonction avec 5.
- ➔ La fonction renvoie 25.



Exercice

- 1 Écrire une fonction `moyenne(liste)` qui retourne la moyenne.
- 2 La tester sur `[12, 15, 9, 18]`.
- 3 Gérer le cas d'une liste vide.

CONDITIONS

if**if**

Teste une condition :
si elle est vraie, le bloc s'exécute.

elif**elif**

Ajoute une condition alternative :
si la première est fausse et celle-ci est vraie,
le bloc s'exécute.

else**else**

S'exécute si aucune des conditions
précédentes n'est vraie.

<>

Opérateurs de comparaison

==

!=

<

>

<=

>=

Exemple

```
note = 14
if note >= 16:
    print("Excellent")
elif note >= 10:
    print("Admis")
else:
    print("Insuffisant")
```

- Si note est 16 ou plus → "Excellent"
- Sinon si note est 10 ou plus → "Admis"
- Sinon → "Insuffisant"



EXERCICE

1

Écrire une fonction qui
affiche "pair" ou "impair".

Nombre



"pair"
ou
"impair"

2

La tester sur 3 nombres différents.

Exemples :

- 4 → pair
- 7 → impair
- 15 → impair

3

Ajouter un cas particulier
pour 0.

Pour 0, afficher :
"0 n'est ni pair ni impair"

LES BOUCLES EN PYTHON



Une boucle permet d'exécuter plusieurs fois les mêmes instructions.

1

BOUCLE FOR



Permet de **parcourir** chaque élément d'une séquence (liste, chaîne, tuple, dictionnaire, ensemble, range(), ...) et d'exécuter des instructions pour chaque élément.

À retenir :

→ La boucle **for** est idéale quand on connaît le nombre d'itérations ou qu'on veut parcourir une collection.

Exemple : parcourir une liste

```
1 fruits = ["pomme", "banane", "cerise"]
2 for fruit in fruits:
3     print(fruit)
4                                     # affiche chaque fruit
```

2

BOUCLE WHILE



Permet de répéter des instructions tant qu'une **condition** est vraie.

À retenir :

→ La boucle **while** est utile quand le nombre d'itérations n'est pas connu à l'avance et dépend d'une condition.

Exemple : compte à rebours

```
1 compteur = 3
2 while compteur > 0:
3     print(compteur)
4     compteur -= 1
5 print("Fin !")                                     # s'exécute quand condition fausse
```

INSTRUCTIONS UTILES AVEC LES BOUCLES

range()

Génère une séquence de nombres.
Très utilisée avec la boucle **for**.

```
range(5) → 0, 1, 2, 3, 4
```

break

Arrête immédiatement la boucle en cours.

```
if x == 0:
    break
```

continue

Passe à l'itération suivante sans exécuter le reste du corps de la boucle.

```
if x % 2 == 0:
    continue
```

Exercice – Les boucles

1

Boucle for — table de multiplication

Écrire un programme Python qui demande à l'utilisateur de saisir un entier n , puis affiche sa table de multiplication de 1 à 10 à l'aide d'une boucle for.

2

Boucle while — deviner un nombre

Écrire un programme Python qui demande à l'utilisateur de deviner un nombre secret. Tant que le nombre saisi est différent du nombre secret, le programme continue à demander une nouvelle valeur.

- Si le nombre est plus petit que le nombre secret, afficher « Trop petit ! ».
- Si le nombre est plus grand que le nombre secret, afficher « Trop grand ! ».
- Lorsque le nombre est trouvé, afficher « Bravo ! Vous avez trouvé le nombre secret. »

3

Somme des nombres pairs

Écrire un programme Python qui calcule et affiche la somme de tous les nombres pairs compris entre 1 et 50 en utilisant une boucle for et l'instruction continue.

4

Cas particulier : break

Modifier le programme de l'exercice 1 afin d'arrêter la boucle après 5 itérations en utilisant l'instruction break, puis afficher un message indiquant que la boucle s'est arrêtée.

Manipulation de fichiers

open() : ouvrir un fichier

La fonction open() dit à Python quel fichier utiliser, et ce qu'on veut en faire

```
open("data.txt", "r")
```

↑ nom du fichier

↑ mode d'ouverture

On indique toujours DEUX choses : quel fichier, et comment on veut l'utiliser (lire ? écrire ?).

"r"

read — lecture

Pour lire un fichier existant

"w"

write — écriture

Crée le fichier (ou écrase le contenu)

"a"

append — ajout

Ajoute du texte à la fin, sans effacer



read() → récupère TOUT le contenu d'un coup, sous forme d'un seul grand texte.

readlines() → récupère le contenu ligne par ligne, sous forme de liste.

Manipulation de fichiers

with: ne jamais oublier de fermer un fichier

Sans with, il faut penser à appeler `f.close()` soi-même — on l'oublie souvent !

```
with open("data.txt", "r") as f:
    contenu = f.read()
```

```
with open("sortie.txt", "w") as f:
    f.write("Résultat : OK")
```

→ le fichier se ferme **TOUT SEUL** à la fin du bloc with

1

Pourquoi with ?

Il garantit que le fichier se referme automatiquement, même en cas d'erreur pendant le code.

2

as f

"f" est juste un nom de variable qui représente le fichier ouvert — on peut l'appeler autrement.

3

Indentation

Tout ce qui touche le fichier doit être écrit sous le with, avec un retrait (indentation).

Manipulation de fichiers

EXERCICE PRATIQUE · GOOGLE COLAB

Importer et lire un fichier dans Google Colab

Utilisez `files.upload()`, puis comparez la lecture d'un fichier sans et avec l'instruction `with`.

1 Importer le fichier depuis votre ordinateur

Dans Colab, on ne peut pas lire directement un fichier de votre ordinateur : il faut d'abord le téléverser dans le notebook avec `files.upload()`.

```
from google.colab import files
uploaded = files.upload()
```

2 Lire le fichier SANS `with`

```
f = open(nom_fichier, "r")
contenu = f.read()
print(contenu)
f.close()

# il faut penser à fermer soi-même
```

3 Lire le fichier AVEC `with`

```
with open(nom_fichier, "r") as f:
    contenu = f.read()
    print(contenu)
```

4 Qu'est-ce que vous remarquez ?

- Dans quelle version avez-vous dû appeler `close()` vous-même ? Dans laquelle est-ce automatique ?
- Que se passerait-il si une erreur survenait pendant la lecture, dans chaque version — le fichier resterait-il ouvert ?
- Laquelle des deux écritures est la plus courte et la plus sûre à utiliser en pratique ?

Manipulation de fichiers

Fiche mémo

Les 5 choses à retenir sur la manipulation de fichiers

```
open(fichier, mode)
```

Ouvre un fichier : "r" pour lire, "w" pour écrire (écrase), "a" pour ajouter

```
with ... as f:
```

Ferme le fichier automatiquement à la fin, même en cas d'erreur

```
f.read()
```

Récupère tout le contenu en un seul bloc de texte

```
f.readlines()
```

Récupère le contenu sous forme de liste, une entrée par ligne

```
pd.read_csv()
```

En IA / data science, la méthode la plus utilisée pour charger des données